

- 数据库管理软件 设计文档
 - 一、项目概述
 - 1.1 项目背景
 - 1.2 核心功能
 - 1.3 目标用户
 - 二、技术架构
 - 2.1 技术栈
 - 2.2 架构模式
 - 三、核心模块设计
 - 3.1 认证模块 (Auth Module)
 - 3.2 数据库连接管理模块 (Connection Module)
 - 3.3 SQL执行模块 (Execution Module)
 - 3.4 AI辅助模块 (AI Module)
 - 3.5 Schema管理模块 (Schema Module)
 - 3.6 查询结果处理模块 (Result Module)
 - 3.7 SQL历史模块 (History Module)
 - 四、数据流转和交互流程
 - 4.1 用户认证流程
 - 4.2 数据库连接管理流程
 - 4.3 SQL执行流程
 - 4.4 智能填表流程
 - 4.5 Markdown文档解析流程
 - 4.6 查询结果展示和导出流程
 - 五、数据库设计 (内置SQLite元数据库)
 - 5.1 db_connections (数据库连接配置表)
 - 5.2 ai_configs (AI配置表)
 - 5.3 sql_history (SQL执行历史表)
 - 5.4 sql_snippets (SQL片段表)
 - 5.5 ai_conversations (AI对话历史表)
 - 5.6 markdown_uploads (Markdown上传记录表)
 - 5.7 数据加密说明
 - 六、前端设计
 - 6.1 设计理念
 - 6.2 主题配色 (浅色主题)
 - 6.3 字体系统
 - 6.4 状态管理设计 (Pinia)
 - 6.5 Monaco Editor配置

- 6.4 核心页面结构
- 6.6 组件设计
- 6.7 响应式设计
- 六A、术语和缩略语
- 六B、功能优先级
 - P0（必须有 - MVP）
 - P1（重要 - 核心价值）
 - P2（一般 - 增强体验）
 - P3（可选 - 未来扩展）
- 六C、系统约束和限制
 - 技术约束
 - 安全约束
 - 业务约束
- 六D、验收标准
 - 功能完整性
 - 性能标准
 - 安全标准
 - 兼容性标准
 - 可用性标准
 - 代码质量标准
- 七、API接口设计
 - 7.1 认证接口
 - 7.2 数据库连接接口
 - 7.3 SQL执行接口
 - 7.4 AI接口
 - 7.5 Schema接口
 - 7.6 结果导出接口
 - 7.7 历史和片段接口
 - 7.8 AI配置接口
- 八、安全设计
 - 8.1 认证与授权
 - 8.2 数据加密
 - 8.3 SQL注入防护
 - 8.4 操作安全
- 九、性能优化
 - 9.1 数据库连接优化
 - 9.2 查询结果优化
 - 9.3 AI调用优化

- 9.4 前端优化
- 十、部署方案
 - 10.1 后端部署
 - 10.2 前端部署
 - 10.3 数据库初始化
- 十一、开发计划
 - 11.1 阶段划分
 - 11.2 技术风险
- 十二、附录
 - 12.1 技术选型说明
 - 12.2 未来扩展

数据库管理软件 设计文档

文档版本: 1.0

创建日期: 2026-06-10

作者: AI辅助设计

一、项目概述

1.1 项目背景

开发一款支持AI辅助的数据库管理软件，通过网页端访问和管理SQL数据库，并集成大语言模型实现自然语言转SQL、智能数据填充等功能。该软件将与现有系统集成，通过JWT token进行身份认证。

1.2 核心功能

- 数据库连接管理：**用户可自定义添加PostgreSQL和SQLite数据库连接
- SQL执行引擎：**支持手动编写SQL和AI辅助生成SQL两种方式
- AI智能辅助：**
 - 自然语言转SQL（对话式）
 - 智能填表（INSERT新数据）
 - 智能填充NULL值（UPDATE已有数据）

- Markdown文档解析并提取数据

4. **专业SQL编辑器**：Monaco Editor，支持语法高亮、智能提示、SQL片段管理

5. **查询结果处理**：表格展示、复杂类型解析（JSON/YAML/XML）、导出（CSV/Excel）、EXPLAIN分析

1.3 目标用户

- **技术人员**：开发人员、DBA，需要高效的数据库操作工具
- **业务人员**：数据分析师、运营人员，依赖AI辅助进行数据操作

二、技术架构

2.1 技术栈

后端：

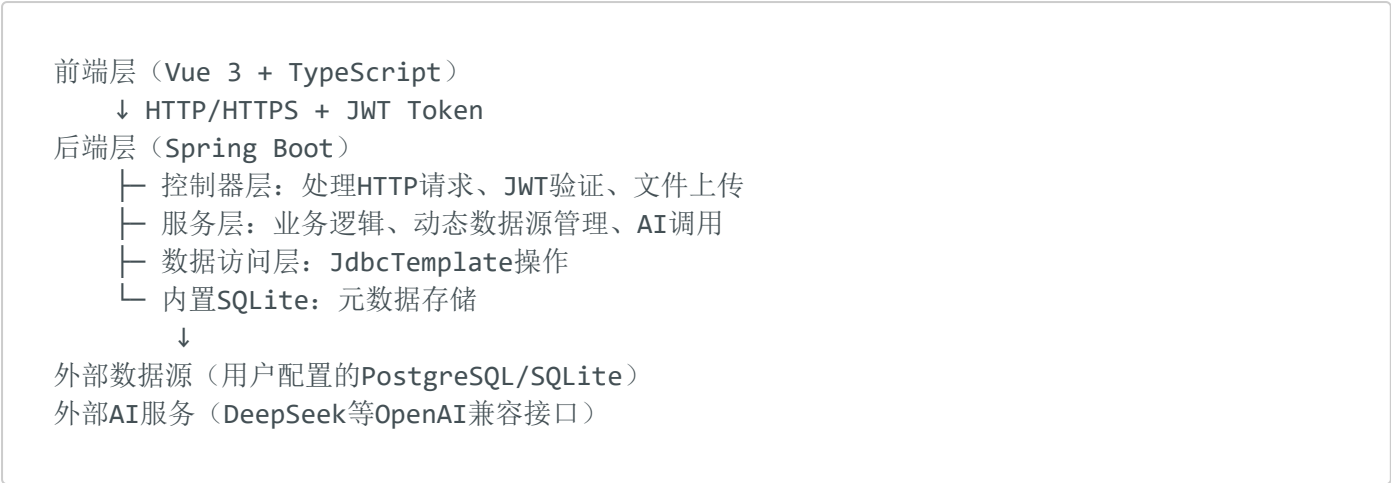
- Java 21
- Spring Boot 3.2.5
- Spring Security 3.2.5（JWT验证）
- Spring JDBC 3.2.5（多数据源管理）
- SQLite 3.45.3.0（内置元数据库）
- HikariCP（连接池）

前端：

- Vue 3.4.31
- TypeScript 5.3.3
- Vite 5.3.3
- Element Plus 2.7.6
- Monaco Editor 0.55.1
- Pinia 2.1.7（状态管理）
- Axios 1.7.2

2.2 架构模式

采用单体Spring Boot应用架构：



优势：

- 与现有系统技术栈一致
- 部署简单（单一jar包）
- 开发效率高，易于维护
- Spring JDBC天然支持多数据源

三、核心模块设计

3.1 认证模块（Auth Module）

功能：

- JWT token解析和验证（从现有系统传入）
- 从token中提取userId
- 支持开发调试模式（dev profile）

开发调试模式：

- 配置项：`app.auth.debug-mode=true` 或使用 `dev` profile
- 提供简易入口，输入userId直接进入系统
- 生产模式下强制JWT验证

3.2 数据库连接管理模块（Connection Module）

功能：

- 用户数据库连接配置的CRUD
- 连接信息加密存储（AES-256）
- 动态数据源管理（AbstractRoutingDataSource）
- 连接池管理（HikariCP）
- 支持PostgreSQL和SQLite两种驱动

SQLite双重用途说明：

1. 内置元数据库（Meta SQLite）

- 路径：{应用目录}/data/metadata.db
- 用途：存储用户配置、连接信息、AI配置、历史记录等
- 权限：应用内部使用，用户不可见
- 初始化：应用启动时自动创建

2. 用户业务数据库（User SQLite）

- 路径：用户指定或上传
- 用途：用户通过连接管理添加的SQLite数据库文件
- 权限：用户可读写、执行SQL
- 管理：作为普通数据库连接管理

连接参数：

- 连接名称
- 数据库类型（postgresql/sqlite）
- 主机地址、端口（PostgreSQL）
- 数据库名/文件路径（SQLite）
- 用户名、密码（加密存储）

动态数据源并发隔离机制：

采用ThreadLocal + 用户上下文的隔离方案：

```
// 伪代码示例
public class DynamicDataSourceHolder {
    private static final ThreadLocal<String> contextHolder = new ThreadLocal<>();

    public static void setDataSource(String userId, String connectionId) {
        // 格式: userId:connectionId
        contextHolder.set(userId + ":" + connectionId);
    }

    public static String getDataSourceKey() {
```

```
        return contextHolder.get();
    }

    public static void clear() {
        contextHolder.remove();
    }
}
```

关键设计：

- 每个用户的每个连接都有唯一的DataSource实例
- 连接池隔离：不同用户的连接不共享连接池
- 请求级别的上下文切换：每次API请求根据JWT token + connectionId定位数据源
- 空闲超时自动释放：超过30分钟未使用的连接池自动关闭

3.3 SQL执行模块（Execution Module）

功能：

- SQL语句解析和执行
- 支持执行全部/选中/当前SQL
- 结果集处理和格式化
- 执行计划分析（EXPLAIN）
- 事务管理
- 危险操作检测（DELETE/DROP/TRUNCATE）并二次确认

执行模式：

- 手动编写SQL执行
- AI生成SQL后执行

3.4 AI辅助模块（AI Module）

功能：

1. AI配置管理

- 用户自定义API Key、API地址、模型名称
- 配置保存在服务器端，自动记忆
- 支持多个OpenAI兼容服务商（DeepSeek、OpenAI、通义千问等）

2. 自然语言转SQL（对话式）

- 用户输入自然语言描述
- 结合当前数据库schema调用AI生成SQL
- 支持多轮对话优化
- SQL返回编辑器供用户修改

3. 智能填表（INSERT）

- 用户选择目标表
- 输入描述性文字
- AI解析并自动填充表单字段
- 用户确认后生成INSERT语句

4. 智能填充NULL值（UPDATE）

- 用户从查询结果中选择有NULL值的记录
- 输入描述性文字
- AI根据已有数据和描述推测NULL字段的值
- 预览后生成UPDATE语句

5. Markdown文档解析

- 用户上传Markdown文件
- 选择目标表
- AI提取文档中的结构化数据
- 预览提取结果（可编辑）
- 批量生成INSERT语句（事务模式）

AI Prompt设计：

1. 自然语言转SQL Prompt

System：你是一个SQL专家，根据用户的自然语言描述生成SQL语句。

当前数据库类型：{dbType}

数据库Schema：

{schemaContext}

用户描述：{userPrompt}

要求：

1. 只返回SQL语句，不要解释
2. 使用标准SQL语法

3. 对于PostgreSQL，使用其特定语法特性
4. 确保SQL语句可以直接执行

2. 智能填表Prompt

System: 你是数据提取专家，从用户描述中提取结构化数据。

目标表: {tableName}

字段定义:

{fields}

用户描述:

{userText}

要求:

1. 以JSON格式返回，格式: {"fieldName": "value", ...}
2. 只提取能从描述中明确获得的字段值
3. 无法确定的字段返回null
4. 日期格式统一为 YYYY-MM-DD
5. 数字类型不要加引号

3. 智能填充NULL Prompt

System: 你是数据推理专家，根据已有数据和用户描述推测缺失字段值。

表结构: {tableSchema}

已有数据: {existingData}

需要填充的字段: {nullFields}

用户描述:

{userDescription}

要求:

1. 返回JSON格式: {"field1": "value1", ...}
2. 基于已有数据的上下文和用户描述进行推理
3. 不确定的字段返回null
4. 保持数据类型一致性

4. Markdown解析Prompt

System: 你是文档数据提取专家，从Markdown文档中提取表格数据。

目标表结构:

{tableSchema}

Markdown内容:

{markdownContent}

要求:

1. 返回JSON数组: [{"field1": "value1", ...}, ...]
2. 尽可能多地提取符合表结构的数据
3. 字段名匹配不区分大小写
4. 如果Markdown中有表格, 优先解析表格
5. 如果是列表或段落, 尝试提取结构化信息

AI调用封装:

```
// 伪代码
public class AIService {

    private final RestTemplate restTemplate;
    private final AIConfigRepository configRepo;

    public AIResponse callAI(String userId, String prompt) {
        AIConfig config = configRepo.findByUserId(userId);

        HttpHeaders headers = new HttpHeaders();
        headers.add("Authorization", "Bearer " + decrypt(config.getApiKey()));

        Map<String, Object> request = Map.of(
            "model", config.getModelName(),
            "messages", List.of(
                Map.of("role", "system", "content", systemPrompt),
                Map.of("role", "user", "content", prompt)
            ),
            "temperature", config.getTemperature(),
            "max_tokens", config.getMaxTokens()
        );

        // 超时控制: 30秒
        return restTemplate.postForObject(
            config.getApiBaseUrl() + "/chat/completions",
            new HttpEntity<>(request, headers),
            AIResponse.class
        );
    }
}
```

3.5 Schema管理模块 (Schema Module)

功能:

- 获取数据库表列表
- 获取表结构 (字段名、类型、约束)
- 支持PostgreSQL和SQLite的元数据查询
- 为编辑器提供智能提示数据

Schema查询实现：

1. PostgreSQL Schema查询

```
-- 获取表列表
SELECT table_name,
       (SELECT COUNT(*) FROM table_name) as row_count
FROM information_schema.tables
WHERE table_schema = 'public';

-- 获取表结构
SELECT column_name, data_type, is_nullable,
       column_default,
       (SELECT COUNT(*) FROM information_schema.key_column_usage
        WHERE table_name = ? AND column_name = ?) > 0 as is_primary_key
FROM information_schema.columns
WHERE table_name = ?;
```

2. SQLite Schema查询

```
-- 获取表列表
SELECT name as table_name,
       (SELECT COUNT(*) FROM sqlite_master WHERE type='table') as row_count
FROM sqlite_master
WHERE type='table' AND name NOT LIKE 'sqlite_%';

-- 获取表结构
PRAGMA table_info(table_name);
```

Monaco Editor智能提示集成：

生成Monaco Editor的CompletionItemProvider数据结构：

```
interface SchemaCompletion {
  tables: {
    name: string;
    columns: {
      name: string;
      type: string;
      nullable: boolean;
    }[];
  }[];
  keywords: string[]; // SQL关键字
  functions: string[]; // SQL函数
}
```

缓存策略：

- Schema信息缓存5分钟
- 用户切换连接时刷新缓存
- 支持手动刷新

3.6 查询结果处理模块（Result Module）

功能：

- 结果集分页（前端指定pageSize）
- 复杂数据类型检测 and 解析（JSON/YAML/XML）
- 结果导出（CSV/Excel格式）
- 查询结果可编辑并回写数据库
- 列排序、筛选

复杂类型检测 and 解析：

1. 检测逻辑

```
// 伪代码
public class ComplexTypeDetector {
    public String detectType(String value) {
        if (value == null) return null;

        // JSON检测
        if (value.trim().startsWith("{") || value.trim().startsWith "[")) {
            try {
                new JSONObject(value); // 或 JSONArray
                return "JSON";
            } catch (Exception e) {}
        }

        // XML检测
        if (value.trim().startsWith("<")) {
            try {
                DocumentBuilder.parse(value);
                return "XML";
            } catch (Exception e) {}
        }

        // YAML检测（简单规则）
        if (value.contains(":") && value.contains("\n")) {
            try {
                Yaml.load(value);
                return "YAML";
            } catch (Exception e) {}
        }
    }
}
```

```
        return "TEXT";
    }
}
```

2. 前端渲染器

```
// 复杂类型字段渲染组件
<template>
  <div v-if="type === 'JSON'" class="json-viewer">
    <pre>{{ formatJSON(value) }}</pre>
    <button @click="copyToClipboard">复制</button>
  </div>
  <div v-else-if="type === 'XML'" class="xml-viewer">
    <pre>{{ formatXML(value) }}</pre>
  </div>
  <div v-else-if="type === 'YAML'" class="yaml-viewer">
    <pre>{{ formatYAML(value) }}</pre>
  </div>
  <div v-else>{{ value }}</div>
</template>
```

结果导出实现：

1. CSV导出

- 使用Apache Commons CSV
- UTF-8 BOM编码（Excel兼容）
- 支持大结果集流式写入

2. Excel导出

- 使用Apache POI
- XLSX格式
- 自动列宽调整
- 最大支持1,048,576行

分页策略：

- 默认每页100行
- 支持10/50/100/500/1000可选
- 后端使用LIMIT/OFFSET实现
- 返回总行数用于分页控件

3.7 SQL历史模块（History Module）

功能：

- 记录所有SQL执行历史（语句、耗时、影响行数、状态）
- SQL片段保存和管理（常用SQL模板）
- 历史查询和检索
- 支持复制SQL、重新执行

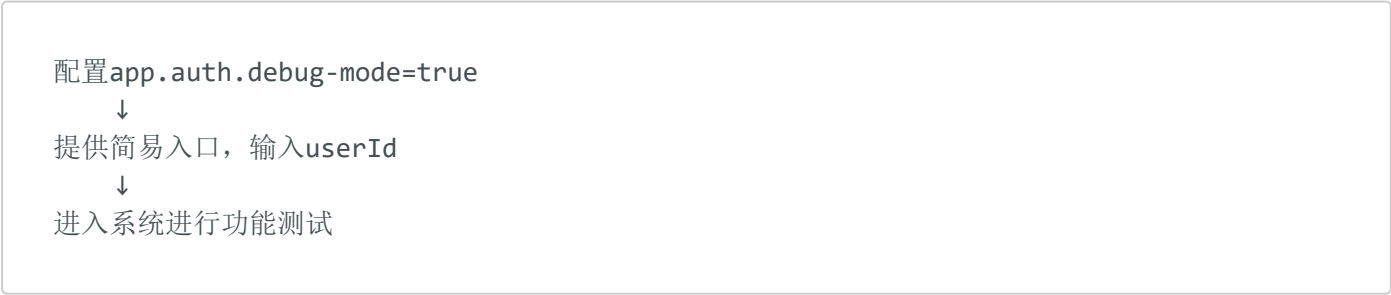
四、数据流转和交互流程

4.1 用户认证流程

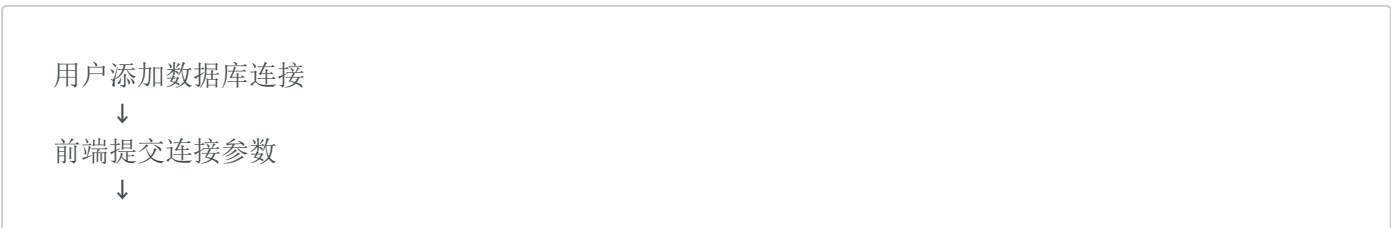
生产模式：



开发调试模式：



4.2 数据库连接管理流程



后端测试连接可用性
↓
使用AES-256加密password
↓
存储到内置SQLite
↓
返回connectionId
↓
用户切换连接 → 动态数据源管理器切换目标数据源

4.3 SQL执行流程

方式A：手动编写SQL

用户在Monaco Editor中编写SQL
↓
编辑器提供：语法高亮、智能提示、格式化
↓
用户点击执行（支持：执行全部/选中/当前）
↓
后端检测危险操作 → 如果是DELETE/DROP/TRUNCATE则返回警告
↓
前端二次确认 → 用户确认后再次请求执行
↓
执行SQL，记录历史，返回结果

方式B：AI辅助生成SQL

用户输入自然语言 → 点击"AI生成SQL"
↓
后端获取当前连接的schema
↓
调用AI API: prompt + schema context
↓
AI返回SQL语句，保存对话历史
↓
SQL显示在Monaco Editor
↓
用户可修改 → 点击执行 → 走方式A流程

4.4 智能填表流程

场景A：新增数据（INSERT）

用户选择目标表 → 点击"新增数据"

↓

前端展示表单（根据表结构生成）

↓

用户输入描述性文字 → 点击"AI解析"

↓

后端调用AI：用户文本 + 字段定义

↓

AI返回结构化数据

↓

前端自动填充表单

↓

用户确认/修改 → 生成INSERT语句并执行

场景B：更新NULL值（UPDATE）

用户执行SELECT查询 → 查看结果表格

↓

选中NULL值的记录 → 点击"智能填充NULL值"

↓

前端发送：主键、已有数据、NULL字段列表、用户描述

↓

后端调用AI：表结构 + 已有数据 + 用户描述

↓

AI返回推测的NULL字段值

↓

前端预览填充结果（高亮显示）

↓

用户确认 → 生成UPDATE语句并执行

4.5 Markdown文档解析流程

用户上传Markdown文件 → 选择目标表

↓

后端解析Markdown内容

↓

获取目标表schema

↓

调用AI：markdown内容 + 表结构

↓

AI返回数组数据

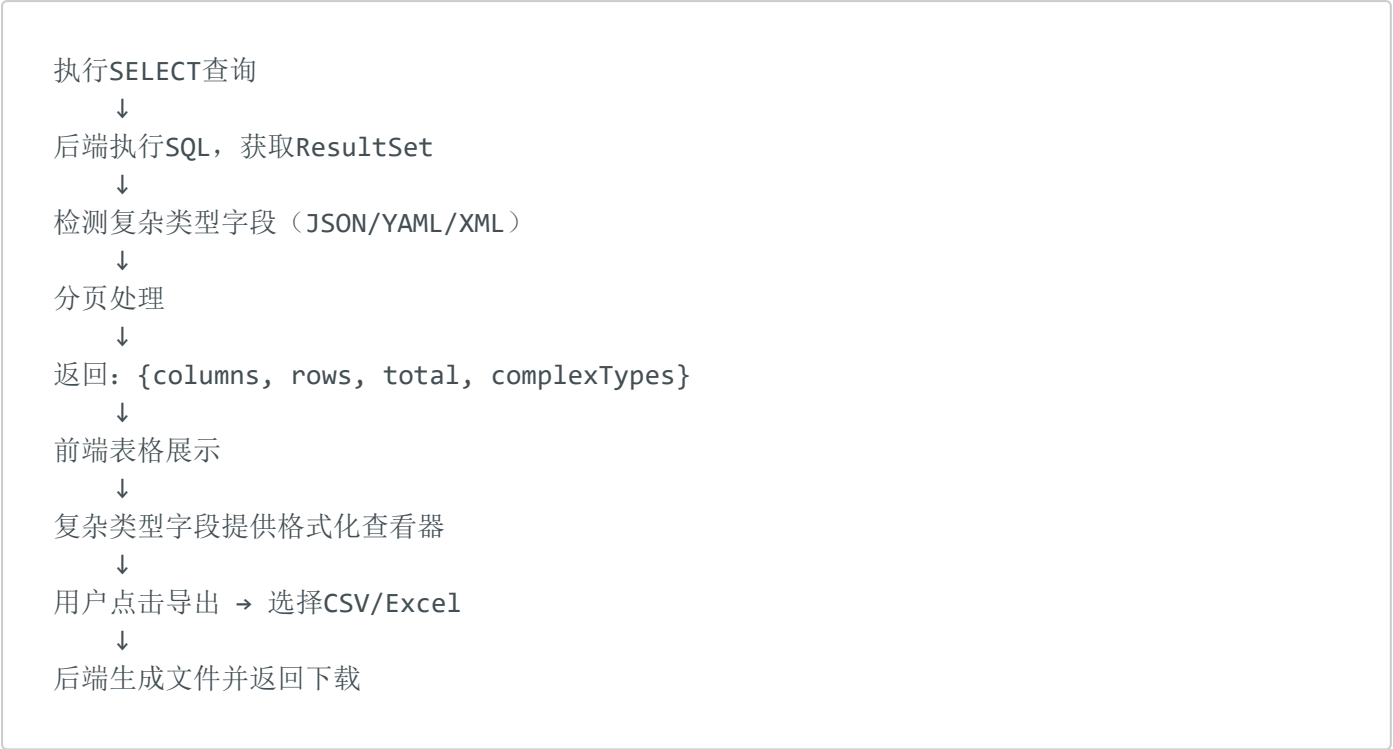
↓

前端展示预览表格（可编辑）

↓

用户确认 → 批量生成INSERT语句（事务模式）并执行

4.6 查询结果展示和导出流程



五、数据库设计（内置SQLite元数据库）

5.1 db_connections（数据库连接配置表）

```
CREATE TABLE db_connections (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id VARCHAR(100) NOT NULL,
  name VARCHAR(200) NOT NULL,
  db_type VARCHAR(20) NOT NULL,           -- postgresql/sqlite
  host VARCHAR(255),
  port INTEGER,
  database_name VARCHAR(200) NOT NULL,
  username VARCHAR(100),
  password_encrypted TEXT,                -- AES-256加密
  is_active BOOLEAN DEFAULT 1,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_connections_user ON db_connections(user_id);
```

5.2 ai_configs（AI配置表）

```
CREATE TABLE ai_configs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id VARCHAR(100) NOT NULL UNIQUE,
  api_provider VARCHAR(50) NOT NULL,          -- deepseek/openai/qwen等
  api_key_encrypted TEXT NOT NULL,            -- AES-256加密
  api_base_url VARCHAR(500),
  model_name VARCHAR(100),
  temperature REAL DEFAULT 0.7,
  max_tokens INTEGER DEFAULT 2000,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5.3 sql_history (SQL执行历史表)

```
CREATE TABLE sql_history (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id VARCHAR(100) NOT NULL,
  connection_id INTEGER NOT NULL,
  sql_text TEXT NOT NULL,
  execution_time INTEGER,                    -- 执行耗时（毫秒）
  rows_affected INTEGER,
  status VARCHAR(20),                        -- success/error
  error_message TEXT,
  executed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (connection_id) REFERENCES db_connections(id)
);
CREATE INDEX idx_history_user_time ON sql_history(user_id, executed_at DESC);
```

5.4 sql_snippets (SQL片段表)

```
CREATE TABLE sql_snippets (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id VARCHAR(100) NOT NULL,
  title VARCHAR(200) NOT NULL,
  sql_content TEXT NOT NULL,
  description TEXT,
  tags VARCHAR(500),                          -- 逗号分隔
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_snippets_user ON sql_snippets(user_id);
```

5.5 ai_conversations (AI对话历史表)

```
CREATE TABLE ai_conversations (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  conversation_id VARCHAR(100) NOT NULL, -- 对话会话ID (UUID)  
  user_id VARCHAR(100) NOT NULL,  
  connection_id INTEGER,  
  message_type VARCHAR(20), -- user/assistant  
  message_content TEXT NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (connection_id) REFERENCES db_connections(id)  
);  
CREATE INDEX idx_conversations_session ON ai_conversations(conversation_id,  
created_at);
```

5.6 markdown_uploads (Markdown上传记录表)

```
CREATE TABLE markdown_uploads (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  user_id VARCHAR(100) NOT NULL,  
  connection_id INTEGER NOT NULL,  
  table_name VARCHAR(200) NOT NULL,  
  file_name VARCHAR(500) NOT NULL,  
  extracted_rows INTEGER, -- 提取的行数  
  inserted_rows INTEGER, -- 成功插入的行数  
  status VARCHAR(20), -- success/partial/error  
  error_message TEXT,  
  uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (connection_id) REFERENCES db_connections(id)  
);  
CREATE INDEX idx_uploads_user ON markdown_uploads(user_id, uploaded_at DESC);
```

5.7 数据加密说明

加密字段：

- `password_encrypted` (数据库连接密码)
- `api_key_encrypted` (AI API密钥)

加密方案：

- 算法：AES-256-CBC
- 密钥管理：密钥存储在配置文件中（application.yml）
- 初始化向量（IV）：与密文一起存储，格式：IV:密文

六、前端设计

6.1 设计理念

Clean Data × Surgical Precision - 医疗级的清洁感与数据可视化的精确性结合

6.2 主题配色（浅色主题）

```
:root {  
  /* 背景层次 */  
  --bg-primary: #FFFFFF;           /* 纯白背景 */  
  --bg-secondary: #F8F9FA;        /* 次级背景 */  
  --bg-tertiary: #F1F3F5;         /* 卡片/面板 */  
  --bg-elevated: #FFFFFF;          /* 悬浮元素（带阴影） */  
  
  /* 文本层次 */  
  --text-primary: #1A1D24;        /* 主要文本 */  
  --text-secondary: #6B7280;      /* 次要文本 */  
  --text-tertiary: #9CA3AF;       /* 辅助文本 */  
  
  /* 边框 */  
  --border-subtle: #E5E7EB;       /* 边框 */  
  --border-default: #D1D5DB;      /* 边框 */  
  --border-strong: #9CA3AF;       /* 边框 */  
  
  /* 强调色 - 专业清晰 */  
  --accent-primary: #0066FF;      /* 主操作 - 鲜明蓝 */  
  --accent-success: #00C853;      /* 成功状态 - 绿 */  
  --accent-warning: #FF9500;      /* 警告 - 橙 */  
  --accent-danger: #FF3B30;       /* 危险操作 - 红 */  
  --accent-info: #00A8E8;         /* 信息 - 青蓝 */  
  
  /* 代码配色（基于 GitHub Light） */  
  --code-keyword: #D73A49;        /* 代码关键字 */  
  --code-string: #032F62;         /* 代码字符串 */  
  --code-number: #005CC5;         /* 代码数字 */  
  --code-function: #6F42C1;       /* 代码函数 */  
  --code-comment: #6A737D;        /* 代码注释 */  
  --code-bg: #F6F8FA;            /* 代码背景 */  
  
  /* 阴影系统 */  
  --shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.04);
```

```
--shadow-md: 0 2px 8px rgba(0, 0, 0, 0.08);
--shadow-lg: 0 4px 16px rgba(0, 0, 0, 0.12);
--shadow-xl: 0 8px 32px rgba(0, 0, 0, 0.16);
--shadow-focus: 0 0 0 3px rgba(0, 102, 255, 0.12);
}
```

6.3 字体系统

- 界面字体：IBM Plex Sans
- 代码字体：JetBrains Mono

6.4 状态管理设计（Pinia）

```
// stores/connection.ts
export const useConnectionStore = defineStore('connection', {
  state: () => ({
    connections: [] as Connection[],
    currentConnection: null as Connection | null,
    schema: null as Schema | null,
  }),

  actions: {
    async fetchConnections() {
      const res = await api.get('/api/connections');
      this.connections = res.data.connections;
    },

    async switchConnection(connectionId: string) {
      const res = await api.post(`/api/connections/${connectionId}/switch`);
      this.currentConnection = this.connections.find(c => c.id === connectionId);
      this.schema = res.data.schema;
    },
  },
});

// stores/sql.ts
export const useSQLStore = defineStore('sql', {
  state: () => ({
    tabs: [] as SQLTab[],
    activeTabId: null as string | null,
    history: [] as SQLHistory[],
    snippets: [] as SQLSnippet[],
  }),

  actions: {
    addTab(content = '') {
      const tab = {
        id: nanoid(),
        name: `query-${this.tabs.length + 1}.sql`,
      };
    },
  },
});
```

```

        content,
        isDirty: false,
    };
    this.tabs.push(tab);
    this.activeTabId = tab.id;
},

updateTabContent(tabId: string, content: string) {
    const tab = this.tabs.find(t => t.id === tabId);
    if (tab) {
        tab.content = content;
        tab.isDirty = true;
    }
},

});

// stores/ai.ts
export const useAIStore = defineStore('ai', {
    state: () => ({
        config: null as AIConfig | null,
        conversations: {} as Record<string, AIMessage[]>, // conversationId -> messages
        isGenerating: false,
    }),

    actions: {
        async generateSQL(prompt: string, conversationId?: string) {
            this.isGenerating = true;
            try {
                const res = await api.post('/api/ai/generate-sql', {
                    connectionId: useConnectionStore().currentConnection?.id,
                    prompt,
                    conversationId,
                });

                // 保存对话历史
                const convId = res.data.conversationId;
                if (!this.conversations[convId]) {
                    this.conversations[convId] = [];
                }
                this.conversations[convId].push(
                    { role: 'user', content: prompt },
                    { role: 'assistant', content: res.data.sql }
                );

                return res.data;
            } finally {
                this.isGenerating = false;
            }
        },
    },
});

// stores/result.ts
export const useResultStore = defineStore('result', {
    state: () => ({
        currentResult: null as QueryResult | null,
    })
});

```

```

    isLoading: false,
    error: null as string | null,
  })),

  actions: {
    async executeSQL(sql: string) {
      this.isLoading = true;
      this.error = null;
      try {
        const res = await api.post('/api/sql/execute', {
          connectionId: useConnectionStore().currentConnection?.id,
          sqlText: sql,
          confirmed: false,
        });

        if (res.data.isDangerous) {
          // 显示二次确认对话框
          const confirmed = await showConfirmDialog(res.data.warningMessage);
          if (confirmed) {
            // 重新执行，带confirmed标识
            const res2 = await api.post('/api/sql/execute', {
              connectionId: useConnectionStore().currentConnection?.id,
              sqlText: sql,
              confirmed: true,
            });
            this.currentResult = res2.data.result;
          }
        } else {
          this.currentResult = res.data.result;
        }
      } catch (err) {
        this.error = err.message;
      } finally {
        this.isLoading = false;
      }
    },
  },
});

```

6.5 Monaco Editor配置

```

// composables/useMonacoEditor.ts
export function useMonacoEditor(containerRef: Ref<HTMLElement>) {
  const connectionStore = useConnectionStore();
  let editor: monaco.editor.IStandaloneCodeEditor;

  onMounted(() => {
    editor = monaco.editor.create(containerRef.value, {
      value: '',
      language: 'sql',
      theme: 'vs',
      fontSize: 14,
    });
  });
}

```

```

    fontFamily: 'JetBrains Mono, monospace',
    minimap: { enabled: false },
    scrollBeyondLastLine: false,
    automaticLayout: true,
    tabSize: 2,
    wordWrap: 'on',
  });

// 注册智能提示
monaco.languages.registerCompletionItemProvider('sql', {
  provideCompletionItems: (model, position) => {
    const word = model.getWordUntilPosition(position);
    const range = {
      startLineNumber: position.lineNumber,
      endLineNumber: position.lineNumber,
      startColumn: word.startColumn,
      endColumn: word.endColumn,
    };

    const suggestions = [];

    // SQL关键字
    const keywords = ['SELECT', 'FROM', 'WHERE', 'INSERT', 'UPDATE', 'DELETE',
'JOIN', 'ORDER BY', 'GROUP BY'];
    keywords.forEach(kw => {
      suggestions.push({
        label: kw,
        kind: monaco.languages.CompletionItemKind.Keyword,
        insertText: kw,
        range,
      });
    });

    // 表名
    if (connectionStore.schema) {
      connectionStore.schema.tables.forEach(table => {
        suggestions.push({
          label: table.tableName,
          kind: monaco.languages.CompletionItemKind.Class,
          insertText: table.tableName,
          range,
          detail: `Table (${table.rowCount} rows)`,
        });
      });

      // 字段名
      table.columns?.forEach(col => {
        suggestions.push({
          label: `${table.tableName}.${col.name}`,
          kind: monaco.languages.CompletionItemKind.Field,
          insertText: col.name,
          range,
          detail: `${col.type} ${col.nullable ? 'NULL' : 'NOT NULL'}`,
        });
      });
    }
  }
});

```



```

        return { suggestions };
    },
});

// 快捷键: Ctrl+Enter执行
editor.addCommand(monaco.KeyMod.CtrlCmd | monaco.KeyCode.Enter, () => {
    const selection = editor.getSelection();
    const sql = selection && !selection.isEmpty()
        ? editor.getModel().getValueInRange(selection)
        : editor.getValue();

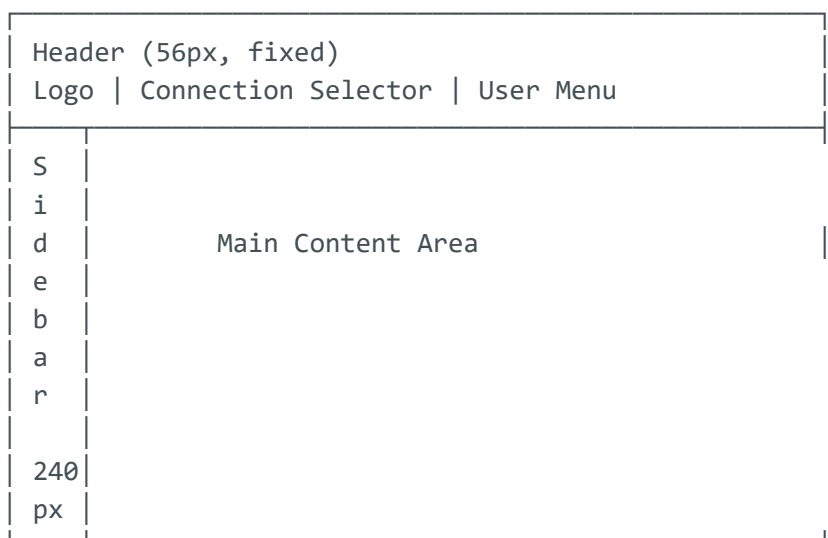
    useResultStore().executeSQL(sql);
});

return { editor };
}

```

6.4 核心页面结构

主布局：



页面列表：

1. SQL编辑器页面（主页面）

- 工具栏：执行/格式化/AI生成/保存片段/执行历史/EXPLAIN
- 编辑器区：Monaco Editor + 多标签页
- AI侧边栏：对话历史 + 输入框（可折叠）
- 结果面板：表格展示 + 导出 + 编辑模式

2. 数据库连接管理页面

- 连接卡片网格
- 新增/编辑连接表单
- 连接状态显示

3. 智能填表页面

- 左侧：文本描述输入区 + AI解析按钮
- 右侧：动态生成表单 + AI填充标识

4. Markdown解析页面

- 文件上传区
- 目标表选择
- 提取结果预览（可编辑表格）
- 批量插入按钮

5. 执行历史页面

- 筛选器：日期/连接/状态
- 历史记录列表
- 详情展开 + 复制SQL + 重新执行

6. 设置页面

- AI配置标签页：API提供商/Key/模型/参数
- 通用设置标签页：编辑器配置/查询配置/安全选项

6.6 组件设计

核心组件列表：

1. **SQLEditor.vue** - SQL编辑器组件

- 集成Monaco Editor
- 多标签页管理
- 快捷键绑定
- SQL格式化

2. **AIAssistant.vue** - AI助手侧边栏

- 对话历史展示
- 消息输入
- SQL应用到编辑器
- 新建对话

3. ResultTable.vue - 查询结果表格

- 虚拟滚动（处理大数据集）
- 列排序、筛选
- 复杂类型渲染
- 可编辑单元格
- 导出功能

4. ComplexTypeViewer.vue - 复杂类型查看器

- JSON格式化展示
- XML格式化展示
- YAML格式化展示
- 语法高亮

5. ConnectionCard.vue - 数据库连接卡片

- 连接信息展示
- 状态指示
- 快捷操作

6. SmartForm.vue - 智能表单

- 动态字段生成
- AI填充标识
- 字段验证
- 预览SQL

6.7 响应式设计

- 最小支持宽度：1280px（桌面应用）
 - 侧边栏可折叠
 - 编辑器和结果区域高度可调整
 - 断点：1280px / 1440px / 1920px
-

六A、术语和缩略语

术语	全称	说明
JWT	JSON Web Token	基于JSON的开放标准令牌，用于身份认证
CRUD	Create Read Update Delete	增删改查操作
AI	Artificial Intelligence	人工智能
LLM	Large Language Model	大语言模型
SQL	Structured Query Language	结构化查询语言
API	Application Programming Interface	应用程序编程接口
AES	Advanced Encryption Standard	高级加密标准
EXPLAIN	SQL查询执行计划	分析SQL查询的执行过程
Schema	数据库模式	数据库的结构定义
Monaco Editor	微软开源代码编辑器	VS Code使用的编辑器核心
HikariCP	高性能JDBC连接池	Java数据库连接池实现
Pinia	Vue 3状态管理库	Vue官方推荐的状态管理方案

六B、功能优先级

P0（必须有 - MVP）

- 用户认证（JWT验证 + 开发调试模式）
- 数据库连接管理（PostgreSQL + SQLite）
- SQL手动编辑和执行
- 基础查询结果展示（表格 + 分页）
- 危险操作二次确认

P1（重要 - 核心价值）

- AI自然语言转SQL
- AI智能填表（INSERT）
- AI智能填充NULL（UPDATE）
- SQL执行历史记录
- Monaco Editor智能提示（schema感知）

P2（一般 - 增强体验）

- Markdown文档解析
- 复杂类型解析（JSON/YAML/XML）
- 结果导出（CSV/Excel）
- SQL片段管理
- EXPLAIN执行计划分析

P3（可选 - 未来扩展）

- 查询结果可视化（图表）
- ER图自动生成
- SQL性能优化建议
- 多人协作功能
- 数据迁移工具

六C、系统约束和限制

技术约束

1. 数据库支持

- PostgreSQL 10.0+
- SQLite 3.x

2. 浏览器兼容性

- Chrome 90+
- Edge 90+
- Firefox 88+
- Safari 14+（有限支持）

3. 并发限制

- 推荐同时在线用户： ≤ 100
- 单用户最大数据库连接数：10
- 单次查询结果建议： $\leq 10,000$ 行（超过则强制分页）

4. 文件限制

- Markdown文件上传： $\leq 10\text{MB}$
- 查询结果导出：CSV无限制，Excel $\leq 1,048,576$ 行

5. 性能指标

- SQL执行响应时间： $< 2\text{s}$ （1000行以内结果集）
- AI生成SQL响应时间： $< 10\text{s}$
- 页面首次加载： $< 3\text{s}$
- 查询结果渲染： $< 1\text{s}$ （100行）

安全约束

1. 认证要求

- 生产环境强制JWT验证
- Token有效期由现有系统控制
- 开发模式仅限开发环境启用

2. 数据加密

- 敏感数据（密码、API Key）必须加密存储
- 传输层使用HTTPS
- AES-256-CBC加密算法

3. 操作限制

- DELETE/DROP/TRUNCATE操作前端二次确认
- 用户只能访问自己创建的数据库连接

- SQL注入防护（参数化查询）

业务约束

1. AI服务依赖

- 需要用户提供AI API密钥
- 支持OpenAI兼容接口
- AI服务故障时降级为手动模式

2. 数据库权限

- 用户需要有目标数据库的访问权限
 - 不同操作需要不同的SQL权限（SELECT/INSERT/UPDATE/DELETE）
-

六D、验收标准

功能完整性

- ☐ P0功能100%实现并通过测试
- ☐ P1功能90%实现
- ☐ P2功能70%实现

性能标准

- ☐ SQL执行（1000行内）：95%请求 <2s
- ☐ AI生成SQL：95%请求 <10s
- ☐ 页面首次加载：<3s
- ☐ 结果表格渲染（100行）：<1s

安全标准

- ☐ JWT验证100%通过
- ☐ 敏感数据加密存储
- ☐ 通过SQL注入测试（OWASP Top 10）

- ☐ 通过XSS测试
- ☐ HTTPS传输

兼容性标准

- ☐ Chrome 90+正常运行
- ☐ Edge 90+正常运行
- ☐ Firefox 88+正常运行
- ☐ 1280px宽度下界面完整显示

可用性标准

- ☐ 用户可以在5分钟内完成首次数据库连接
- ☐ 用户可以在1分钟内通过AI生成并执行SQL
- ☐ 危险操作有明确的警告和二次确认
- ☐ 错误信息清晰易懂，提供解决建议

代码质量标准

- ☐ 单元测试覆盖率 $\geq 70\%$
- ☐ 集成测试覆盖核心流程
- ☐ 代码符合阿里巴巴Java开发手册规范
- ☐ 前端代码通过ESLint检查
- ☐ 无严重级别的安全漏洞

七、API接口设计

7.1 认证接口

POST /api/auth/debug-login（仅开发模式）

- 请求：{userId: string}
- 响应：{token: string}

7.2 数据库连接接口

GET /api/connections

- 响应: `{connections: [{id, name, dbType, host, port, database, isActive, createdAt}]}`

POST /api/connections

- 请求: `{name, dbType, host, port, database, username, password}`
- 响应: `{connectionId, message}`

PUT /api/connections/:id

- 请求: 连接参数
- 响应: `{message}`

DELETE /api/connections/:id

- 响应: `{message}`

POST /api/connections/:id/test

- 响应: `{success: boolean, message}`

POST /api/connections/:id/switch

- 响应: `{message, schema: {tables: [...]}}`

7.3 SQL执行接口

POST /api/sql/execute

- 请求: `{connectionId, sqlText, confirmed: boolean}`
- 响应: `{isDangerous, warningMessage, result: {columns, rows, total, executionTime, rowsAffected}}`

POST /api/sql/explain

- 请求: `{connectionId, sqlText}`
- 响应: `{plan: string}`

7.4 AI接口

POST /api/ai/generate-sql

- 请求: {connectionId, prompt, conversationId?}
- 响应: {sql, conversationId, message}

POST /api/ai/parse-text

- 请求: {connectionId, tableName, textDescription}
- 响应: {fields: {fieldName: value}}

POST /api/ai/fill-nulls

- 请求: {connectionId, tableName, primaryKey, existingData, nullFields, description}
- 响应: {filledFields: {fieldName: value}}

POST /api/ai/parse-markdown

- 请求: multipart/form-data - {file, connectionId, tableName}
- 响应: {extractedData: [{field1: value1, ...}], rowCount}

7.5 Schema接口

GET /api/schema/tables

- 参数: connectionId
- 响应: {tables: [{tableName, rowCount}]}

GET /api/schema/table/:tableName

- 参数: connectionId
- 响应: {columns: [{name, type, nullable, defaultValue, isPrimaryKey}]}

7.6 结果导出接口

POST /api/results/export

- 请求: `{format: 'csv'|'excel', data: {columns, rows}}`
- 响应: 文件流

7.7 历史和片段接口

GET /api/history

- 参数: `connectionId?, startDate?, endDate?, status?`
- 响应: `{history: [{id, sqlText, executionTime, rowsAffected, status, executedAt}]}`

GET /api/snippets

- 响应: `{snippets: [{id, title, sqlContent, description, tags}]}`

POST /api/snippets

- 请求: `{title, sqlContent, description, tags}`
- 响应: `{snippetId, message}`

7.8 AI配置接口

GET /api/ai/config

- 响应: `{apiProvider, apiBaseUrl, modelName, temperature, maxTokens}`

PUT /api/ai/config

- 请求: `{apiProvider, apiKey, apiBaseUrl, modelName, temperature, maxTokens}`
- 响应: `{message}`

八、安全设计

8.1 认证与授权

- 生产模式: 强制JWT验证, 从token提取userId

- **开发模式**：通过配置项启用调试入口
- **会话管理**：JWT token有效期由现有系统控制

8.2 数据加密

- **敏感字段加密**：数据库密码、API密钥使用AES-256-CBC加密
- **密钥管理**：加密密钥存储在配置文件中，不入库
- **传输安全**：HTTPS传输

8.3 SQL注入防护

- 使用参数化查询（PreparedStatement）
- 对用户输入进行验证和转义
- 限制SQL执行权限

8.4 操作安全

- **危险操作检测**：DELETE/DROP/TRUNCATE等操作前端二次确认
 - **操作审计**：所有SQL执行记录到sql_history表
 - **连接隔离**：每个用户只能访问自己创建的连接
-

九、性能优化

9.1 数据库连接优化

- 使用HikariCP连接池
- 动态数据源按需创建，空闲超时自动释放
- 连接池参数可配置

9.2 查询结果优化

- 大结果集分页处理，避免一次性加载
- 流式读取ResultSet

- 前端虚拟滚动渲染大数据表格

9.3 AI调用优化

- AI请求超时控制
- 对话历史限制条数（避免context过长）
- Schema信息缓存（避免重复查询）

9.4 前端优化

- Monaco Editor懒加载
- 路由懒加载
- Element Plus按需引入
- 大文件上传使用分片

十、部署方案

10.1 后端部署

打包方式：

```
mvn clean package -DskipTests
```

运行方式：

```
java -jar database-management-system.jar \
  --spring.profiles.active=prod \
  --server.port=8080
```

配置文件：

- `application.yml`：基础配置
- `application-dev.yml`：开发环境配置
- `application-prod.yml`：生产环境配置

环境变量：

- `AES_SECRET_KEY`：数据加密密钥
- `JWT_SECRET`：JWT验证密钥（与现有系统一致）

10.2 前端部署

构建：

```
npm run build
```

部署方式：

- Nginx静态托管
- 与后端部署在同一域名下，避免跨域问题

10.3 数据库初始化

内置SQLite数据库自动初始化：

- 应用启动时检查表是否存在
- 不存在则自动执行DDL脚本创建表

十一、开发计划

11.1 阶段划分

第一阶段：基础功能（2-3周）

- 认证模块（JWT + 调试模式）
- 数据库连接管理
- SQL手动执行
- 基础查询结果展示

第二阶段：AI功能（2-3周）

- AI配置管理
- 自然语言转SQL
- 智能填表（INSERT）
- 智能填充NULL（UPDATE）

第三阶段：高级功能（1-2周）

- Markdown文档解析
- SQL片段管理
- 执行历史
- 复杂类型解析（JSON/YAML/XML）
- 结果导出（CSV/Excel）

第四阶段：优化和测试（1周）

- 性能优化
- 安全加固
- 集成测试
- 用户体验优化

11.2 技术风险

1. **AI API稳定性**：依赖外部AI服务，需处理超时和失败
 2. **多数据源管理**：动态数据源切换的并发安全性
 3. **大结果集处理**：超大查询结果的内存和性能问题
-

十二、附录

12.1 技术选型说明

为什么选择Spring JDBC而不是MyBatis/JPA？

- 动态数据源管理更灵活
- SQL执行更直接，无ORM映射开销
- 适合数据库管理工具场景

为什么选择内置SQLite而不是MySQL/PostgreSQL？

- 部署简单，无需额外数据库服务
- 元数据量小，SQLite性能足够
- 与应用打包在一起，便于分发

12.2 未来扩展

1. **支持更多数据库类型**：MySQL、SQL Server、Oracle
2. **数据可视化**：查询结果图表展示
3. **协作功能**：SQL片段分享、团队协作
4. **权限管理**：细粒度的数据库操作权限控制
5. **ER图生成**：根据schema自动生成ER图
6. **数据迁移工具**：跨数据库数据同步

文档结束